JavaScript From Scratch

Easy Slide Notes with Basic Functions, DOM Examples, Problems and Solutions

These notes are written in a simple way so a beginner can read them and understand what JavaScript is, how it works, and how to solve basic problems with it.

1. What is JavaScript?

JavaScript is a programming language used to make web pages interactive. HTML makes the structure of a page, CSS makes it beautiful, and JavaScript makes it work.

- HTML creates a button.
- CSS designs the button.
- JavaScript runs an action when the button is clicked.

```
<button onclick="alert('Hello JavaScript!')">
    Click Me
</button>
```

> When the user clicks this button, JavaScript shows a popup message. This is a simple example of interactivity.

2. Ways to Write JavaScript

2.1 Inline JavaScript

Inline JavaScript is written directly inside an HTML tag. It is easy for small testing, but not recommended for big projects.

```
<button onclick="alert('Button clicked!')">Click Me</button>
```

2.2 Internal JavaScript

Internal JavaScript is written inside the <script> tag in the same HTML file.

```
<!DOCTYPE html>
<html>
<body>
    <h1>Hello JavaScript</h1>

    <script>
        alert("Welcome to my website!");
    </script>
</body>
</html>
```

2.3 External JavaScript

External JavaScript is written in a separate .js file. This is the best method for real projects because HTML and JavaScript stay separate and clean.

```
<!-- HTML file -->
<script src="script.js"></script>

// script.js file
alert("This is external JavaScript");
```

3. JavaScript Output Methods

Method	Use
alert()	Shows a popup message in the browser.
console.log()	Shows output in the browser console for testing.
document.write()	Writes directly on the page, mostly for learning.

innerHTML	Changes HTML content using JavaScript.

```
alert("Hello World");
console.log("Testing JavaScript");
document.write("Hello from JavaScript");
```

```
<p id="demo">Old Text</p>
<script>
    document.getElementById("demo").innerHTML = "New Text";
</script>
```

console.log() is very important for beginners because it helps you check values and find mistakes in your code.

4. Variables

Variables are used to store data. In JavaScript, we commonly use let and const.

```
let age = 20;
const country = "Pakistan";

age = 21; // allowed
// country = "India"; // not allowed because country is const
```

Keyword	Explanation
var	Old way to create variables. Avoid it in modern JavaScript.
let	Use when the value can change later.
const	Use when the value should not change.

5. Data Types

Data type means the kind of value stored in a variable.

Data Type	Example	Meaning
String	"Ali"	Text value
Number	25, 99.5	Numeric value
Boolean	true / false	Yes or no condition
Undefined	let x;	Variable created but value not assigned
Null	let user = null;	Empty value intentionally assigned
Array	["Apple", "Mango"]	Multiple values in one variable
Object	{name:"Ali", age:20}	Data in key-value pairs

```
let name = "Ali";
let age = 20;
let isLogin = true;
let fruits = ["Apple", "Mango", "Banana"];
let student = { name: "Ali", age: 20, city: "Lahore" };

console.log(student.name); // Ali
```

6. Operators

6.1 Arithmetic Operators

Arithmetic operators are used for math operations.

```
let a = 10;
let b = 5;

console.log(a + b); // 15
console.log(a - b); // 5
console.log(a * b); // 50
console.log(a / b); // 2
console.log(a % b); // 0 remainder
```

6.2 Comparison Operators

Comparison operators compare values and return true or false.

```
console.log(10 > 5);   // true
console.log(10 < 5);   // false
console.log(10 >= 10); // true
console.log(10 !== 5); // true
```

6.3 == and ===

```
console.log(5 == "5");  // true, checks only value
console.log(5 === "5"); // false, checks value and data type
```

> In real projects, use === because it is safer and avoids unexpected bugs.

6.4 Logical Operators

```
let age = 20;
let hasCNIC = true;

console.log(age >= 18 && hasCNIC === true); // AND
console.log(age >= 18 || hasCNIC === true); // OR
console.log(!hasCNIC); // NOT
```

7. Conditions

Conditions are used when we want the program to make decisions.

7.1 if else

```
let age = 16;

if (age >= 18) {
    console.log("You can vote");
} else {
    console.log("You cannot vote");
}
```

7.2 else if

```
let marks = 75;

if (marks >= 90) {
    console.log("Grade A+");
} else if (marks >= 80) {
    console.log("Grade A");
} else if (marks >= 70) {
    console.log("Grade B");
```

```
} else {
    console.log("Grade C");
}
```

7.3 switch

```
let day = 3;

switch (day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    case 3:
        console.log("Wednesday");
        break;
    default:
        console.log("Invalid day");
}
```

switch is useful when we have many fixed options, like days, roles, status values, or menu choices.

8. Loops

Loops are used to repeat code again and again.

8.1 for Loop

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

This prints numbers from 1 to 5.

8.2 while Loop

```
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

8.3 do while Loop

```
let i = 1;

do {
    console.log(i);
    i++;
} while (i <= 5);
```

A do while loop runs at least one time, even if the condition is false.

9. Functions

A function is a reusable block of code. We write a function once and use it many times.

9.1 Basic Function

```
function sayHello() {
    console.log("Hello World");
```

```
}

sayHello();
```

9.2 Function With Parameters

```
function greet(name) {
    console.log("Hello " + name);
}

greet("Ali");
greet("Ahmed");
```

9.3 Function With Return

```
function add(a, b) {
    return a + b;
}

let result = add(10, 5);
console.log(result); // 15
```

9.4 Arrow Function

```
const add = (a, b) => {
    return a + b;
};

console.log(add(5, 3));

// Short form
const multiply = (a, b) => a * b;
```

10. Basic Useful Functions

10.1 Add Two Numbers

```
function addNumbers(a, b) {
    return a + b;
}

console.log(addNumbers(10, 20)); // 30
```

10.2 Check Even or Odd

```
function checkEvenOdd(number) {
    if (number % 2 === 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

console.log(checkEvenOdd(10)); // Even
console.log(checkEvenOdd(7));  // Odd
```

10.3 Find Greater Number

```
function findGreater(a, b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

```
console.log(findGreater(10, 20)); // 20
```

10.4 Square of a Number

```
function square(number) {
    return number * number;
}

console.log(square(5)); // 25
```

10.5 Celsius to Fahrenheit

```
function celsiusToFahrenheit(celsius) {
    return (celsius * 9 / 5) + 32;
}

console.log(celsiusToFahrenheit(30)); // 86
```

10.6 Positive, Negative, or Zero

```
function checkNumber(num) {
    if (num > 0) {
        return "Positive";
    } else if (num < 0) {
        return "Negative";
    } else {
        return "Zero";
    }
}
```

11. Arrays

An array stores multiple values in one variable. Array index starts from 0.

```
let fruits = ["Apple", "Mango", "Banana"];

console.log(fruits[0]); // Apple
console.log(fruits[1]); // Mango
console.log(fruits.length); // 3
```

Common Array Methods

Method	Use
push()	Adds an item at the end.
pop()	Removes the last item.
shift()	Removes the first item.
unshift()	Adds an item at the start.
length	Gives total number of items.

```
let fruits = ["Apple", "Mango"];
fruits.push("Banana");
console.log(fruits);

fruits.pop();
console.log(fruits);
```

Loop Through Array

```
let fruits = ["Apple", "Mango", "Banana"];

for (let i = 0; i < fruits.length; i++) {
    console.log(fruits[i]);
}
```

12. Objects

An object stores data in key-value pairs. It is useful for storing related information about one thing.

```
let student = {
    name: "Ali",
    age: 20,
    city: "Lahore"
};

console.log(student.name);
console.log(student.age);
```

Object With Method

```
let student = {
    name: "Ali",
    age: 20,
    introduce: function() {
        console.log("My name is " + this.name);
    }
};

student.introduce();
```

A function inside an object is called a method. this.name means the name property of the same object.

13. DOM - Connect JavaScript With HTML

DOM stands for Document Object Model. It allows JavaScript to select and change HTML elements.

13.1 Change Text

```
<h1 id="heading">Old Heading</h1>
<button onclick="changeText()">Change Text</button>

<script>
function changeText() {
    document.getElementById("heading").innerHTML = "New Heading";
}
</script>
```

13.2 Change Color

```
<p id="para">This is a paragraph.</p>
<button onclick="changeColor()">Change Color</button>

<script>
function changeColor() {
    document.getElementById("para").style.color = "red";
}
</script>
```

13.3 Hide and Show Element

```
<p id="text">This text will hide.</p>
<button onclick="hideText()">Hide</button>
<button onclick="showText()">Show</button>
```

```
<script>
function hideText() {
    document.getElementById("text").style.display = "none";
}
function showText() {
    document.getElementById("text").style.display = "block";
}
</script>
```

14. Events

Events happen when a user interacts with a web page.

Event	Use
onclick	Runs when an element is clicked.
onmouseover	Runs when mouse goes over an element.
onmouseout	Runs when mouse leaves an element.
onchange	Runs when input value changes.
onkeyup	Runs when keyboard key is released.
onsubmit	Runs when form is submitted.

```
<button onclick="sayHello()">Click Me</button>

<script>
function sayHello() {
    alert("Hello!");
}
</script>
```

15. Form Validation Example

```
<form onsubmit="return validateForm()">
    <input type="text" id="name" placeholder="Enter your name">
    <button type="submit">Submit</button>
</form>

<p id="error"></p>

<script>
function validateForm() {
    let name = document.getElementById("name").value;

    if (name === "") {
        document.getElementById("error").innerHTML = "Name is required";
        return false;
    }

    alert("Form submitted successfully");
    return true;
}
</script>
```

This function checks if the name field is empty. If empty, it shows an error and stops the form. If filled, it allows submission.

16. Basic Problems With Solutions

Problem 1: Add Two Numbers

```
function add(a, b) {
    return a + b;
}

console.log(add(5, 10)); // 15
```

Problem 2: Check Even or Odd

```
function evenOdd(num) {
    if (num % 2 === 0) {
        return "Even";
    } else {
        return "Odd";
    }
}

console.log(evenOdd(8)); // Even
```

Problem 3: Largest of Three Numbers

```
function largestOfThree(a, b, c) {
    if (a >= b && a >= c) {
        return a;
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}

console.log(largestOfThree(10, 25, 15)); // 25
```

Problem 4: Factorial of a Number

Factorial of 5 means 5 x 4 x 3 x 2 x 1 = 120.

```
function factorial(num) {
    let result = 1;

    for (let i = 1; i <= num; i++) {
        result = result * i;
    }

    return result;
}

console.log(factorial(5)); // 120
```

Problem 5: Print Even Numbers From 1 to 20

```
for (let i = 1; i <= 20; i++) {
    if (i % 2 === 0) {
        console.log(i);
    }
}
```

Problem 6: Sum of Numbers From 1 to 10

```
let sum = 0;

for (let i = 1; i <= 10; i++) {
    sum = sum + i;
```

```
}

console.log(sum); // 55
```

Problem 7: Reverse a String

```
function reverseString(str) {
    let reversed = "";

    for (let i = str.length - 1; i >= 0; i--) {
        reversed = reversed + str[i];
    }

    return reversed;
}

console.log(reverseString("hello")); // olleh
```

Problem 8: Check Palindrome

A palindrome is a word that reads the same forward and backward, like madam.

```
function checkPalindrome(str) {
    let reversed = "";

    for (let i = str.length - 1; i >= 0; i--) {
        reversed = reversed + str[i];
    }

    if (str === reversed) {
        return "Palindrome";
    } else {
        return "Not Palindrome";
    }
}

console.log(checkPalindrome("madam")); // Palindrome
```

Problem 9: Find Maximum Number in an Array

```
function findMax(numbers) {
    let max = numbers[0];

    for (let i = 1; i < numbers.length; i++) {
        if (numbers[i] > max) {
            max = numbers[i];
        }
    }

    return max;
}

console.log(findMax([10, 50, 30, 90, 20])); // 90
```

Problem 10: Sum of Array Numbers

```
function sumArray(numbers) {
    let sum = 0;

    for (let i = 0; i < numbers.length; i++) {
        sum = sum + numbers[i];
    }

    return sum;
}

console.log(sumArray([10, 20, 30, 40])); // 100
```

Problem 11: Count Vowels

```
function countVowels(str) {
    let count = 0;
    let vowels = "aeiouAEIOU";

    for (let i = 0; i < str.length; i++) {
        if (vowels.includes(str[i])) {
            count++;
        }
    }

    return count;
}

console.log(countVowels("JavaScript")); // 3
```

Problem 12: Simple Calculator Function

```
function calculator(a, b, operator) {
    if (operator === "+") {
        return a + b;
    } else if (operator === "-") {
        return a - b;
    } else if (operator === "*") {
        return a * b;
    } else if (operator === "/") {
        return a / b;
    } else {
        return "Invalid operator";
    }
}

console.log(calculator(10, 5, "+")); // 15
```

17. Mini Project: Counter App

```
<!DOCTYPE html>
<html>
<body>
    <h1 id="count">0</h1>

    <button onclick="increase()">Increase</button>
    <button onclick="decrease()">Decrease</button>
    <button onclick="reset()">Reset</button>

    <script>
        let count = 0;

        function increase() {
            count++;
            document.getElementById("count").innerHTML = count;
        }

        function decrease() {
            count--;
            document.getElementById("count").innerHTML = count;
        }

        function reset() {
            count = 0;
            document.getElementById("count").innerHTML = count;
        }
    </script>
</body>
</html>
```

count stores the current number. increase() adds 1, decrease() subtracts 1, and reset() sets count back to 0.

18. Mini Project: Simple Calculator

```
<!DOCTYPE html>
<html>
<body>
    <h2>Simple Calculator</h2>

    <input type="number" id="num1" placeholder="First number">
    <input type="number" id="num2" placeholder="Second number">

    <button onclick="calculate('+')">Add</button>
    <button onclick="calculate('-')">Subtract</button>
    <button onclick="calculate('*')">Multiply</button>
    <button onclick="calculate('/')">Divide</button>

    <h3 id="result">Result: </h3>

    <script>
        function calculate(operator) {
            let num1 = Number(document.getElementById("num1").value);
            let num2 = Number(document.getElementById("num2").value);
            let result;

            if (operator === "+") result = num1 + num2;
            else if (operator === "-") result = num1 - num2;
            else if (operator === "*") result = num1 * num2;
            else if (operator === "/") result = num1 / num2;

            document.getElementById("result").innerHTML = "Result: " + result;
        }
    </script>
</body>
</html>
```

Number() converts input text into numbers. The calculate() function checks which button was clicked and then shows the result on the page.

19. Important JavaScript Summary

Topic	Use
alert()	Shows popup message.
console.log()	Shows output in console.
let	Creates a variable that can change.
const	Creates a variable that cannot change.
array	Stores multiple values.
object	Stores key-value data.
if else	Makes decisions.
for loop	Repeats code.
function	Reusable block of code.
return	Sends value back from function.
document.getElementById()	Selects HTML element by ID.

innerHTML	Changes HTML content.
onclick	Runs code on click.

20. Beginner Tips

- Start with variables, data types, operators, conditions, loops, and functions.
- Use console.log() to test and debug your code.
- Use let and const instead of var.
- Use === instead of == for safer comparisons.
- Practice DOM examples because DOM connects JavaScript with HTML.
- Build small projects like counter, calculator, form validation, and todo list.